The Observed-Information Program

Gene Wen · jw9263@nyu.edu

A research program playbook covering one law, one theorem, and a shared cost model across language, serving, and model structure.

Working draft, July 2026. Internal planning document with detailed execution instructions.

0. How to read this document

This playbook brings seven ongoing projects together into a single research program and gives step by step instructions to carry it out. Sections 1 to 3 state the thesis, the flagship theorem, and the unifying law. Sections 4 to 6 are the operational core; for each of three domains they give concrete experimental protocols, controls, expected outcomes, and go or no-go criteria. Sections 7 to 12 cover methodology, the twelve month timeline, risks, deliverables, and positioning against prior work. The appendices fix notation, the mutual information estimation protocol, a reproducibility checklist, and a full run sheet for the pivotal experiment. Every instruction is written so a collaborator can run it without needing the source papers.

0.1 The seven projects at a glance

Project	In one line	Role in the program
Forward–Backward POE	Product-of-experts fusion of a forward and a backward LM for token correction	Core: the theorem and its evidence
Admission control	Gate requests on a calibrated latency tail rather than a point estimate	Core: the systems instance
FuseCorrect	Repair a pruned layer segment with a causal token-graph corrector	Core: the intra-model instance
SphereMerge	Merge redundant layers by debiased directional geometry	Support: localization
BackSlash	Rate-constrained training that codes LLM weights near their true information rate	Support: training-time weight coding
AUGUR	Spec-first, uncertainty-aware LLM performance estimator	Support: the cost and economics layer
HOSVD	Higher-order SVD of weight-training trajectories	Foundational, off-axis appendix

1. Thesis and the information ledger

Across all seven projects one principle keeps returning. Only observed information changes a decision. Self-generated or purely modelled information, however plausible it looks, carries no incremental value. We call the bookkeeping of this principle the information ledger. Every computation a system performs falls into one of three lines. There is information it observes, whether from the world, a tool, a sensor, or an independently trained source. There is information it generates from what it already has, which is free but inert. And there is the physical cost of moving whatever information it uses.

An efficient system therefore has three jobs, and the program is organized around them. First, locate where the real, observed information is. Second, pay only the physical cost of moving that information. Third, never spend computation treating generated information as if it were observed. The first job is a question of information availability, answered by the flagship theorem and its instances. The second is a cost question, answered by AUGUR. The third is a discipline we call the honesty budget, enforced by calibration, grounding labels, and regime maps throughout.

The bet is that this ledger is measurable rather than metaphorical, and that one inequality governs phenomena that currently look unrelated: whether a language model can correct itself, whether a serving gate should admit a request, and whether a pruned network layer can be repaired. If the bet holds, the program is a single lens rather than seven separate papers.

2. The flagship theorem (No Free Information)

2.1 Statement

Let x<t denote the observed context, x_t the current token, and f any deterministic function of the context. Then the conditional mutual information vanishes: I(x_t ; f(x<t) | x<t) = 0. In plain terms, no self-generated continuation adds any information about the current token beyond what the context already holds. This covers chain-of-thought, self-critique, best-of-N sampling from the same model, and iterative self-refinement.

2.2 Proof sketch

Condition on x<t. Because f is a deterministic function of x<t, its value is fixed once x<t is fixed, and a constant shares zero mutual information with anything. The result follows from the data-processing inequality applied to the Markov chain x_t, x<t, f(x<t). Under stochastic decoding with temperature above zero, f is no longer deterministic, but the bound still holds in expectation over the sampling noise, because that noise is independent of x_t given x<t. The temperature control in Section 4.2 verifies this empirically.

2.3 What breaks the bound, and why it matters

Two things break the bound, and pinning them down is where the practical payoff lives. The first is an external observation, such as a tool result, a unit-test execution, or a retrieval hit. It injects information that is not a function of x<t, so its conditional mutual information with x_t can be positive. The second is an independently parameterized model. From the joint distribution over data and parameters, that model is not a deterministic function of x<t, so its predictions can carry non-zero information. That non-zero quantity is exactly what Section 4.2 sets out to measure. Every recommendation in this document follows from the split between these two cases. Spend compute on observation and on independent signal, and never on self-generation.

2.4 Corollaries to state explicitly in the paper

Corollary 1, the self-improvement ceiling. Without an external channel, no amount of self-generated reasoning improves local token correctness. This is a clean negative result in the open debate over whether models can improve themselves. Corollary 2, directionality is irreducible. A backward model that reads the observed future supplies information no amount of additional forward context can supply, because the future tokens are observations rather than functions of the past. Corollary 3, capacity is not the scarce resource. Since the gain comes from directionality rather than model size, a small backward model can correct an arbitrarily large forward model. Section 4.3 tests this claim.

2.5 A worked example (why this is not obvious)

Consider a model that has produced a chain-of-thought and is now asked to critique its own answer. The critique feels informative, since it is new text the model had not written before. The theorem says it is not. The critique is a deterministic function of the same context that produced the answer, so once we condition on that context it is fixed and shares zero information with the correct token. What feels like new evidence is only surface novelty. Now run the generated code against a test suite. The pass or fail bit is not computable from the context alone, because it depends on the external interpreter, so it carries real information and can change the correction. The program is the disciplined pursuit of that difference.

3. The unifying law and its seven instances

The same inequality governs every project. Each one either locates observed information, prices its movement, or refuses to fabricate it. Figure 1 is the program map. The table that follows states, for each project, the observed information it exploits and the generated or modelled thing it must not trust.

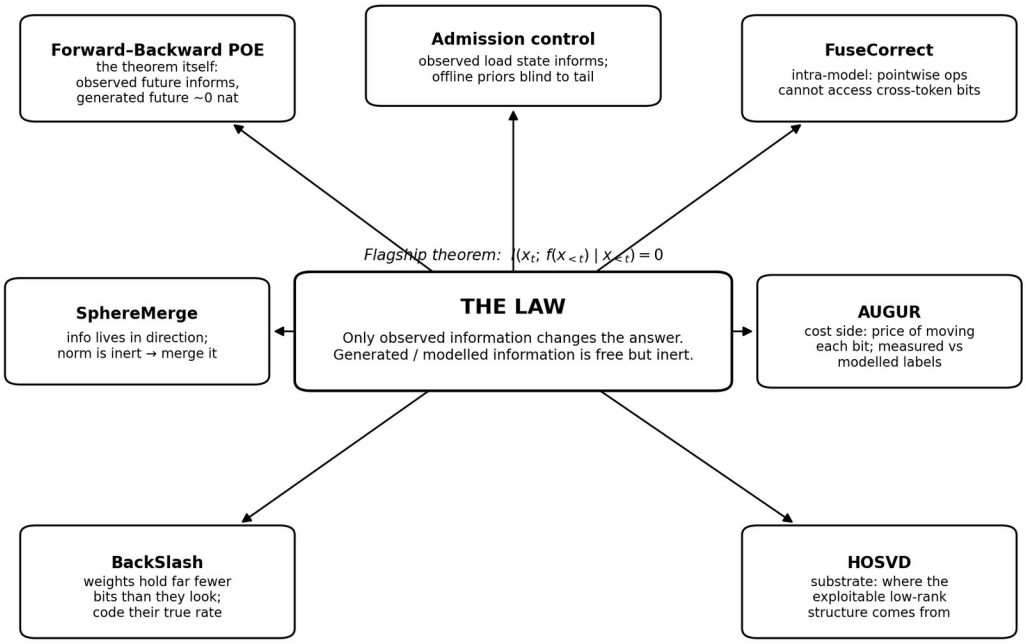

Figure 1. One law, seven instances. Every arrow reads as "is an instance of."

Project	**Observed information it exploits**	**Generated or modelled thing it must not trust**
Forward–Backward POE	The observed future tokens x>t, or tool and test results	A self-predicted future from the same model (~0 nat)
Admission control	The observed serving state (queue depth at admission)	An offline latency prior blind to the queueing tail
AUGUR	Measured calibration rows and hardware specifications	An unlabeled extrapolation presented as if measured
FuseCorrect	Cross-token structure accessible in the boundary window	A pointwise fit pretending to recreate token mixing
SphereMerge	The directional component of the representation	The magnitude or norm, which is inert for the task
BackSlash	The true information rate of the weights, a heavy-tailed generalized-Gaussian source	The nominal parameter count or bit-width as a proxy for information
HOSVD	The low-rank subspace shared across trajectories	Off-axis directions with no cross-task structure

4. Domain 1: language and correction (Forward–Backward POE)

This domain owns the theorem and its sharpest evidence. It has four workstreams: generalize the theorem, run the independent-training experiment that pins down the break condition, establish scale invariance, and demonstrate the law in a real deployment where the future is naturally observed.

4.1 Generalize the theorem

Step 1. Rewrite the current Section 4 result, which is stated only for predicted tokens, in terms of an arbitrary deterministic function f of x<t (Section 2.1). Step 2. Enumerate the important instances of f and state each as a named corollary: chain-of-thought traces, self-critique, Constitutional-AI-style revisions, best-of-N from the same model, and iterative self-refinement. Step 3. Prove the stochastic-decoding extension in expectation form and cite the temperature control (4.2) as its empirical confirmation. Step 4. Write the two break conditions as a positive design rule. Information helps only when it is an external observation or an independently parameterized source. Deliverable: a self-contained theorem section with three corollaries.

4.2 Independent-training experiment E1–E4

Purpose. Locate the exact edge of the bound. A same-model prediction carries zero information (E1), the true observed future is the information ceiling (E3, the oracle), and the POE-trained backward model is the method (E4). The critical unknown is E2, how much a genuinely independent model contributes, because E4 minus E2 is the single number that shows directionality beats mere model diversity.

Setup. Train two GPT-2 124M models with identical architecture and data (WikiText-103) but different seeds for both initialization and data order. Model A uses seed 42, model B uses seed 137, each trained for 100k steps with matched hyperparameters. Confirm genuine independence by checking that the held-out logits correlation falls between 0.70 and 0.90. If it sits above that range the models are too correlated, so re-seed; if it falls below, re-check the training.

Measurement. For each test position t, form the fused log-probability $\log p_f(\cdot|x<t) + \alpha \cdot \log p_{pred}(\cdot|future)$ with $\alpha = 0.9$, and record the KL gain over the forward-only distribution at the true token. Average over positions to get the mutual information in nats. Run the four conditions in the matrix below.

Expt	Predictor of the future	Expected MI	Purpose
E1	Model A predicts its own future	≤ 0.0019 nat	Reproduce the DPI zero baseline
E2	Model B (independent) predicts	> 0.0019, < 0.368 nat	Quantify model-diversity information
E3	True observed future x>t (Oracle)	≈ 0.368 nat	Information ceiling from observation
E4	POE-trained backward model	≈ 0.662 nat	The method's realized gain

Interpretation matrix. Read the four comparisons below before writing any conclusion. The E4 minus E2 gap is the paper's single most important number.

Key comparison	Result meaning	Action
E2 − E1 ≈ 0	Model diversity adds no information	Attribute the gain to directionality (strongest)
E2 − E1 > 0	Diversity carries some information	Quantify its share, still compare against E4
E4 − E2 ≫ 0	Directionality beats diversity	Core claim holds, lead with it
E4 − E2 ≈ 0	Gain may be mostly diversity	Re-examine, reframe honestly if it replicates
E3 − E2 ≫ 0	Diversity cannot replace observation	Observed future is irreplaceable

Controls, run all three. Temperature sweep: repeat E1 at T in {0.0, 0.5, 1.0, 1.5} and confirm the mutual information stays near zero, which closes the objection that stochastic decoding evades the bound. Independent-model scale sweep: run E2 with predictors of 45M, 124M, 355M, and 1.5B to separate independence from capacity. Row-shuffle sanity check: randomly permute model B's logits across positions and

confirm the estimate collapses to about zero, which shows the estimator is not leaking. Decision rule: if E2 minus E1 is about zero the backward gain is purely directional, the strongest outcome; if E4 minus E2 stays large, directionality dominates diversity and the core claim holds.

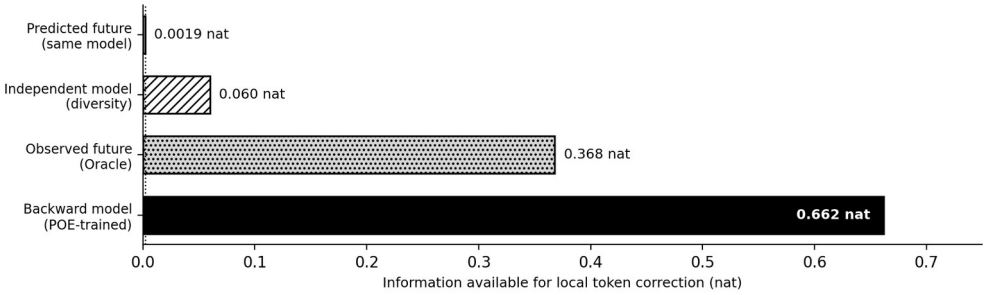

Figure 2. Measured spine of the theorem, per error position. The independent-model bar is what E2 will pin down, shown here as an illustrative mid-range value.

4.3 Scale-invariance experiment (7B/13B forward)

Purpose. Show that a fixed micro backward model corrects an arbitrarily large forward model. Step 1. Hold the backward model fixed at 45M. Step 2. Sweep the forward model across 124M, 355M, 774M, 1.5B, then 7B and 13B, recording the retained fraction of the backward advantage at each size. It already sits near 80% from 124M to 1.5B, and about 101% at 45M against 124M internally. Step 3. Fit the retention curve and report whether it plateaus. Go criterion: retention stays above about 70% at 7B and 13B. The headline if it holds: directionality, not capacity, is the scarce resource, and a tiny fixed corrector becomes a strong efficiency argument for edge deployment.

4.4 Deployment with a naturally observed future (code generation)

Purpose. Turn the theory into a systems result by finding a setting where the observed future comes for free. The primary target is code generation on HumanEval, where executing the unit tests is a genuine external observation that breaks the bound. Step 1. Generate a candidate completion. Step 2. Execute the tests and feed the pass or fail outcome and any error traces to the corrector as the observed signal. Step 3. Run end to end generate then correct, and report accuracy against a no-correction baseline and against same-model self-refinement, which the theorem predicts should not help. Secondary targets are tool-call returns and verifier checks on multi-step math (GSM8K). Deliverable: one external task-accuracy number that turns the impossibility theorem into a working system.

5. Domain 2: serving and control (Admission + AUGUR)

This domain shows the same inequality in systems form and supplies the cost side of the ledger.

5.1 Load-aware admission gate

Motivation. The current admission result is validated only in the easy regime: per-token decode, 0/1 cost, static load, homogeneous workload. There a calibrated scalar margin equals the full predictive distribution. The open problem is load. Above saturation, end to end latency is 90 to 95% queue wait, and every offline-calibrated gate is blind to it. Step 1. Build a continuous-batching serving loop driven by measured decode-versus-batch curves under Poisson and bursty (MMPP) arrivals on RTX 4090 and H200. Step 2. Sweep the arrival rate through the saturation knee, expected near 11 to 13 requests per second, and confirm the knee where violation and queue-wait fraction jump together. Step 3. Compare offline gates (point, margin, distribution) against a gate that conditions on the observed queue depth at admission. Expected outcome, stated as the law: offline priors admit about 100% and violate 89 to 97% above the knee, and only the observed-state gate holds. Step 4. Confirm the knee location and tail magnitude against a real vLLM run. Deliverable: a load-aware gate that proves the inequality in practice. Observed serving state carries the tail information, and modelled priors do not.

5.2 AUGUR as the economics layer

Role. AUGUR prices the movement of each bit and labels every number as measured, partially measured, or modelled. This is the honesty budget in numerical form. Step 1. Wire AUGUR's sub-millisecond per-operation estimate in as the cost term, so the program's decision rule becomes: act only when value exceeds cost, where value is the observed-information gain and cost is AUGUR's bandwidth-bound price. Step 2. Close the two headline AUGUR gaps that gate its credibility. The first is the cold-start benchmark against spec-only simulators (LLMCompass, LIFE, LIMINAL, LLM-Viewer) on a common corpus. The second is the set of modifier rows that are modelled only today, namely KV-cache compression, speculative decoding, and continuous batching. Step 3. Keep the bandwidth-hull guard. When a target falls outside the calibration hull, fall back to a physics estimate with wide intervals rather than an overconfident correction. Deliverable: AUGUR positioned as the economics that makes the program operational, not as a separate project.

6. Domain 3: model structure (FuseCorrect, SphereMerge, BackSlash)

This domain shows the availability bound inside a network: what cross-token structure a corrector can access, where in the representation the task information actually lives, and how much information the weights truly carry.

6.1 FuseCorrect

Claim to establish. A deleted transformer segment removes cross-token routing that a pointwise corrector cannot recreate by construction. Only an operator with access to cross-token structure can repair it. Step 1. Delete a contiguous K-layer segment, for example 7 layers of Llama-3.1-8B, chosen by a baseline-compatible selector. Step 2. Insert the two-branch corrector. An affine branch with closed-form initialization restores channel alignment, then a causal token-graph branch builds a sparse graph over a local causal window from learned feature distances and runs a few unrolled graph-diffusion steps. Step 3. Enforce the matched-budget protocol. Every method shares the deleted segment, the calibration data, and the evaluation suite, compared at equal parameter budget against training-free LinearPatch, an LLM-Streamline-style MLP replacement, and the affine-only ablation. Step 4. Quantify the availability floor. Show that the best pointwise map still leaves about 49% of the delete-only boundary error, and that about 42% of the segment's response energy is genuinely cross-token. Deliverables: fill in the headline perplexity and multiple-choice results and the affine-only versus full delta, and state the result as an availability bound.

6.2 SphereMerge

Claim to establish. Task information lives in the direction of the hidden state, not its norm, and the normalized sphere is contaminated by a few massive-activation components that must be suppressed before scoring redundancy. Step 1. Reproduce the probe result that a direction-only linear probe matches full-activation accuracy, about 80.06% against 80.01%, while a norm-only probe collapses to chance at about 53.44%. Step 2. Confirm that after L2 normalization a single massive-activation principal component explains about 81.2% of variance on LLaMA-3-8B. Step 3. Score redundancy with Debiased Spherical Information Alignment. Project out the top-k principal directions, normalize, and score soft assignments to shared anchors under a normalized-mutual-information criterion with one temperature. Step 4. Fuse with magnitude-preserving weighting and fold in the closed-form Fusion-aware Compensation scalar, so inference cost is unchanged. Report the 42-times scoring speedup over the diffusion-kernel baseline and the clean composition with 4-bit quantization. The framing: pruning removes the inert, information-free axes.

6.3 BackSlash

Claim to establish. A model's weights carry far less information than their bit-width suggests. Trained parameters follow a generalized-Gaussian distribution with shape below 2, so their true rate is a small fraction of the nominal size, and rate-constrained training can drive the model to that floor without losing accuracy. Step 1. Fit the generalized-Gaussian shape parameter to each layer's weights and confirm it sits below 2, for

example about 1.36 for BERT, 1.54 for GPT-2, 1.26 for Llama-3, and 0.85 for DeepSeek. Step 2. Define the discretized generalized-Gaussian rate R(θ) as the mean of $|\theta_i|$ raised to the power of the shape parameter, estimating the shape per batch from the ratio $E[\theta^2]$ over $E[|\theta|]^2$. Step 3. Train with the rate-distortion objective $J = D + \lambda R$, using soft gradient clipping (add a small ε inside the power) to stop the gradient from blowing up when the shape drops below 1. Step 4. Entropy-code the trained weights with exp-Golomb codes at k = 0 after ranking parameter values by frequency, which lands within a few points of the entropy limit and needs no per-model table. Expected outcome: 75 to 90% size reduction with roughly 1 to 2% accuracy change, and models that stay accurate under pruning to 80 to 90% where ordinary training fails past 60%. The framing: measure and pay for the weights' real information rate rather than their nominal precision. As a bonus, the extra sparsity enlarges the headroom that the FuseCorrect and SphereMerge instances exploit.

7. Cross-cutting methodology: the honesty budget

The program's second signature, after the law itself, is that every exploitation comes with an explicit account of its support. Enforce this everywhere. Label every reported number as measured, partially measured, or modelled. Attach calibrated intervals to every predictive claim and report empirical coverage against nominal levels. Publish a regime map for each decision that shows where a cheap statistic suffices and where the full distribution is required. Use one shared mutual-information estimator across all domains (Appendix B) so the language, serving, and structure results are numerically comparable. This discipline is what makes the thesis scientific rather than promotional, and it is the part reviewers will trust.

8. Milestones and twelve-month timeline

The plan front-loads the theorem and the two lowest-risk experiments, independent training and FuseCorrect results, runs the systems and scaling work through the middle, and reserves the last quarter for the deployment result and writing. Figure 3 shows the schedule. The go or no-go gates are: theorem generalized and E1–E4 measured by M4; scale invariance and the load-aware knee reproduced by M7; deployment accuracy by M10.

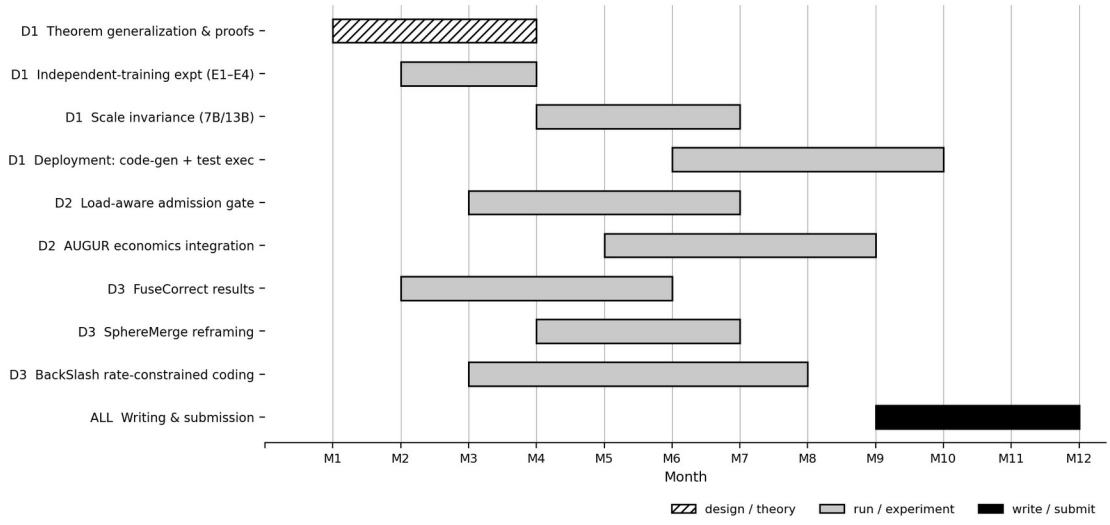

Figure 3. Twelve-month schedule across the three domains. Design and theory, experiment, and writing phases are distinguished by fill.

9. Risks and mitigations

Risk	Why it threatens the program	Mitigation
E2 ≈ Oracle (diversity explains the gain)	Would collapse the directionality claim into ordinary ensembling	Row-shuffle and scale sweeps to detect leakage; if real, reframe as a diversity result and report it honestly

Scale invariance fails above 1.5B	Removes the strongest efficiency headline	Report the retention curve as measured; fall back to the fixed-corrector claim at the scales where it holds
No clean deployment with observed future	Theory stays internal, weaker systems contribution	Prioritize code-gen with test execution; tool-returns and math verifiers as backups
AUGUR cross-family transfer stays weak	Limits the cost layer to near-distribution targets	Use ranking with wide intervals off-support; bandwidth-hull guard; scope claims to the interior
FuseCorrect graph diffusion underperforms	Weakens the intra-model availability instance	Matched-budget protocol still yields an informative floor result even if gains are modest
Reviewers see a forced grand unification	Undercuts the program framing	Keep the core spine tight (theorem, admission, FuseCorrect); mark HOSVD as an off-axis appendix

10. Deliverables, success metrics, and venues

Success is defined by four concrete deliverables, each with a measurable bar. First, the generalized No-Free-Information theorem with three corollaries and the E1–E4 measured spine, including E4 minus E2 as the headline number. Second, a load-aware admission gate that holds above the saturation knee where offline gates fail. Third, the scale-invariance result: the retained backward advantage stays above about 70% at 7B and 13B with a fixed 45M corrector. Fourth, one external task-accuracy number, on HumanEval or GSM8K, that shows end to end correction from an observed future.

Deliverable	**Success metric**	**Target venue**
Generalized theorem + E1–E4	E4 − E2 large and reproducible across seeds	Competitive ICLR / ACL
Load-aware admission gate	Holds above knee; offline gates violate 89–97%	Systems (MLSys / OSDI-style)
Scale invariance (fixed corrector)	≥ 70% retention at 7B/13B	ICLR / EMNLP
Deployment result	Accuracy gain vs self-refinement baseline	EMNLP / ACL findings, strong main-track case

One line. Only observed information changes the answer. One law, one impossibility theorem, and one cost model, tested across language, serving, and model structure.

11. Positioning against prior work

Language and self-improvement. A large body of empirical work debates whether LLMs can self-correct their reasoning, with mixed and often negative findings but no unifying account. The generalized theorem supplies that account. Without an external channel, self-generated feedback carries zero incremental local information, so any method that appears to work must be smuggling in an external signal such as a task reward, a verifier, or a tool. Framed this way, a scattered debate becomes a clean impossibility result together with a recipe, and that is the contribution reviewers will remember.

Serving and estimation. AUGUR sits among performance tools such as Vidur, MIST/HERMES, LLMCompass, and ALA, each strong on one slice. Its distinct position is spec-first coverage with calibrated uncertainty at sub-millisecond cost, which is exactly what an admission gate needs. The admission work is deliberately downstream of estimation. It consumes the calibrated tail and shows that only observed serving state closes the gap above saturation, something no offline estimator can do. The program presents the estimator and the policy as one loop rather than two isolated results.

Compression and structure. Depth-compression methods such as ShortGPT, LLM-Streamline, SLEB, Shortened-LLaMA, and MKA mostly study which layers to remove. FuseCorrect and SphereMerge reframe the field around information availability: what cross-token structure a corrector can access, and where in the representation the task information actually lives. BackSlash extends the same view to training itself. Because trained weights are a low-information, heavy-tailed source, rate-constrained training codes them near their true rate and leaves the model far more prunable, which the other structure instances then exploit. Together these are a lens rather than a marginal metric gain, and it is what ties the compression work to the language and serving instances under one inequality.

12. Compute budget and resourcing

The program is staged so the highest-impact result, the generalized theorem and its E1–E4 spine, is also the cheapest to produce, and the expensive scale work comes only after the core claim is secure. The table below gives an order-of-magnitude budget per workstream. It assumes a small shared cluster and treats the two independent trainings as parallelizable. Read it as a sequencing guide rather than a procurement document. Fund Domain 1 first, and gate the rest on its outcomes.

Workstream	**Approx. compute**	**Note**
Theorem generalization	Negligible (analysis)	Paper and proof work, no training
E1–E4 independent training	2 GPU-trainings × 100k steps	3–5 days wall-clock in parallel
Scale invariance 7B/13B	Inference-only sweeps	Fixed 45M corrector, forward models frozen
Code-gen deployment	Generation + test execution	Compute-light, the interpreter is the observation
Load-aware admission gate	Serving-loop simulation + one vLLM run	Bulk is simulation, one real confirmation run
AUGUR economics + cold-start	Calibration + benchmark rows	Adds SGLang and modifier rows, no large training
FuseCorrect / SphereMerge	Corrector training + scoring	Small correctors, scoring is 42× cheaper than MKA
BackSlash rate-constrained training	One training run with a rate term	Per-batch shape estimate adds minor overhead

Two resourcing principles keep the plan robust. First, no expensive workstream starts before its cheap precondition is met. The 7B/13B sweep waits on the retention curve holding at 1.5B, and the load-aware gate waits on the saturation-knee simulation. Second, every workstream shares one mutual-information estimator and one honesty-budget labeling convention (Appendix B), so results compose without re-tooling and a reviewer can trace any number back to a single logged configuration.

Appendix A: notation

x<t is the observed context up to position t. x_t is the current token. x>t is the observed future. f is an arbitrary deterministic function of the context, that is, a self-generated continuation. p_f and p_b are the forward and backward model distributions. α is the backward exponent in the product-of-experts fusion, default 0.9. $I(\cdot;\cdot|\cdot)$

is conditional mutual information in nats. The oracle ceiling is the information available from the true observed future. E1 to E4 are the four experiments of Section 4.2.

Appendix B: mutual-information estimation protocol

Use one estimator across all domains for comparability. For each evaluation position, compute the forward-only predictive distribution and the fused distribution, which is the forward distribution plus the weighted predictor log-probabilities, then take the KL gain at the observed token. Average over a fixed held-out set and report in nats with a five-seed mean and standard deviation. Always run the row-shuffle null, permuting predictor logits across positions, and confirm the estimate returns to about zero. A non-zero null indicates leakage in the pipeline and invalidates the run. Report the coverage of any predictive intervals against nominal 80, 90, and 95% levels.

Appendix C: reproducibility checklist

Fix and log all seeds for initialization and data order. Record the architecture, parameter count, training steps, and hyperparameters for every model. Store held-out logits and the logits-correlation check for the independence experiments. Version the calibration corpus snapshot for any AUGUR number, and label each value measured, partial, or modelled. Release the serving-loop configuration, meaning the arrival process, rates, and GPU, for the admission experiments. Every headline number should be regenerable from a single logged configuration.

Appendix D: full run sheet for E1–E4

This is the exact sequence a collaborator follows to produce the Figure-2 spine. Estimated wall-clock is 3 to 5 days with the two trainings run in parallel.

Step 1. Train model A, the forward model. Set all seeds, for initialization and data order, to 42; architecture GPT-2 124M (n_embd 768, n_layer 12, n_head 12); data WikiText-103 shuffled with seed 42; 100k steps; save to model_A. Expected time is 1 to 2 days on one GPU.

Step 2. Train model B, the independent predictor. Identical architecture and data, with seeds set to 137 for both initialization and data order; 100k steps; save to model_B. Run it in parallel with Step 1.

Step 3. Verify independence. Compute the correlation of model A and model B held-out logits and require it to fall between 0.70 and 0.90. Above 0.90 the models are too correlated, so re-seed. Below 0.70, investigate under-training or a data mismatch.

Step 4. Run E1. Use model A to predict its own future greedily, fuse with $\alpha = 0.9$, and estimate the mutual information; reproduce ≤ 0.0019 nat. Step 5. Run E2. Use model B as the predictor and record the mutual information. Step 6. Run E3. Use the true observed future x>t and expect about 0.368 nat. Step 7. Run E4. Use the POE-trained backward model and expect about 0.662 nat.

Step 8. Controls. Temperature sweep on E1 at T in {0.0, 0.5, 1.0, 1.5}, expecting about zero throughout. Row-shuffle null on E2, permuting model B logits across positions, expecting the estimate to return to about zero. Optional independent-model scale sweep at 45M, 124M, 355M, and 1.5B. Step 9. Compile. Report E1 to E4 with a five-seed mean and standard deviation, then the two decisive deltas E4 minus E2 and E3 minus E2, and interpret them via the Section 4.2 matrix.

Appendix E: key numbers to reproduce

These are the empirical anchors the program stands on. Treat each as a target to reproduce and, where noted, to extend, and label every one measured, partial, or modelled in the final write-up.

Quantity	Value	Where and status
Self-predicted future (DPI zero)	≤ 0.0019 nat	E1, measured
Observed future (Oracle ceiling)	≈ 0.368 nat	E3, measured

Backward model (POE-trained)	≈ 0.662 nat	E4, measured
Signal within a ≤ 8-token window	≈ 73%	Forward–Backward, measured
Error-detection AUC, ours vs MLM-PLL	0.704 vs 0.551	Forward–Backward, measured
Backward-advantage retention 124M→1.5B	≈ 80%	Scale invariance, extend to 7B/13B
SphereMerge scoring speedup vs MKA	42×	SphereMerge, measured
Direction vs norm linear probe	80.06% vs 53.44%	SphereMerge, measured
Massive-activation PC share of variance	81.2%	SphereMerge (LLaMA-3-8B), measured
Pointwise-repair residual floor	≈ 49%	FuseCorrect, measured
Cross-token share of response energy	≈ 42%	FuseCorrect, measured
Admission saturation knee	≈ 11–13 req/s	Admission, reproduce on real vLLM
Offline-gate violation above knee	89–97%	Admission, target for the load-aware gate
BackSlash size reduction	75–90%	BackSlash, measured (up to 90% on Gemma)
GG weight shape parameter	< 2	BackSlash, measured (0.85 to 1.54)
Prunable ratio after BackSlash	80–90%	BackSlash, measured (vs < 60% normal)
Optimal exp-Golomb parameter	k = 0	BackSlash, measured